

Random Number Generation and Entropy Sources

Revision 2026-10-06 — supersedes the earlier “Random Number Generation and Entropy Sources” note (which described `dd if=/dev/urandom` key generation and used `entropy_avail = 256` as evidence).

Measured on a grey board (shiba-25q3ml), kernel 6.6.116, OpenSSL 3.5.4 + FIPS provider 3.1.2, branch `tpm-spi-encryption`.

1. Key generation method

1.1 shiba_get_random (/usr/lib/shiba/shiba-random.sh)

All key, salt, and PIN material generated by our scripts goes through one function, in two stages:

1. **Entropy-source blocking gate** — a 1-byte read from `/dev/random`. On Linux ≥ 5.6 this returns only once the kernel CRNG is fully seeded (the same `wait_for_random_bytes()` gate behind `get_random_bytes()`). It fails closed after 120 s.
2. **Output from the OpenSSL FIPS DRBG** — `openssl rand`, i.e. the FIPS provider’s SP 800-90A CTR_DRBG (AES-256), seeded from the kernel CRNG.

The output file is created mode `0400`. Short output, gate timeout, or a DRBG error removes the file and returns non-zero. The library is installed in both the rootfs and the initramfs, so first-boot provisioning also waits for the CRNG.

Material	Size	Caller
OTA rootfs LUKS key file	64 B (512-bit)	<code>update-rootfs-luks-verity-with-tpm.sh</code>
FDE submask_B (TPM gate)	64 B	<code>init-fde.sh</code>
FDE salt	32 B	<code>fde-kek-lib.sh</code> (<code>fde_generate_salt</code>)
Grey data KEK	64 B	<code>fde-kek-lib.sh</code> (<code>fde_grey_generate_kek</code>)
FE salt / FE key	32 B / 64 B	<code>fe-lib.sh</code>
VPN PKCS#11 PIN	8 B	<code>vpn-pkcs11-pin.sh</code>

1.2 LUKS volume keys

The LUKS volume (master) keys are generated by `cryptsetup luksFormat`. `cryptsetup 2.7.5` uses the OpenSSL backend in FIPS mode:

```
Crypto backend (OpenSSL 3.5.4 30 Sep 2025 [fips][threads][argon2])
```

1.3 OpenSSL random generators

`openssl list -random-generators` shows CTR-DRBG, HASH-DRBG, and HMAC-DRBG, all from the `fips` provider. The only active providers are `base` and `fips`.

2. Entropy sources

2.1 Kernel CRNG

- Readiness evidence: `random: crng init` done at 5.07 s after boot. This is before the `initramfs` unlocks or provisions LUKS (`dm-crypt` starts at about 10.4 s).
- `/proc/sys/kernel/random/entropy_avail` is not evidence. On Linux ≥ 5.18 it always reads 256, the same as `poolsize`. The earlier note's `entropy_avail = 256` argument does not hold. Readiness is shown by `crng init` done and enforced by the gate in §1.1.

2.2 TPM hardware RNG

- `/sys/class/misc/hw_random/rng_current = tpm-rng-0`.
- `rng_core` quality is 1024 (`current_quality` and `default_quality`). The kernel `hwrng` thread therefore credits TPM output straight into the CRNG; this is what seeds the CRNG early in boot.
- `rngd -f -r /dev/hwrng` also runs (`rng-tools.service`). When the system is idle the kernel issues about one TPM `GetRandom` call (2 TPM commands) per 30 s.
- The kernel mixes the TPM input with timing and interrupt entropy, and the CRNG reseeds periodically.

2.3 Power-on self-tests

- `crypto-selftest` passes RNG (Continuous RNG Test), with `/dev/hwrng` as the noise source.
- `nss-drbg-selftest` passes the softokn FIPS POST (DRBG KAT) and `PK11_GenerateRandom`.

3. TPM bus (SPI) protection

TPM random output and TPM-held secrets cross the SPI bus. Both the kernel path and the user-space path now use TPM 2.0 salted sessions with parameter encryption.

Path	Mechanism	Covers
Kernel → TPM	<code>CONFIG_TCG_TPM2_HMAC</code> (backported to 6.6.116, <code>recipes-kernel/linux/files/6.6/0005-tpm-hmac-sessions.patch</code>). Salt: ECDH P-256 to a null-hierarchy primary. Cipher: AES-128-CFB. MAC: HMAC-SHA-256.	<code>hwrng TPM2_GetRandom</code> (response encrypted + HMAC), <code>TPM2_PCR_Extend</code> (HMAC)
User space → TPM	<code>tpm2-tools</code> salted sessions (<code>tpm_start_encrypted_policy_session</code> / <code>tpm_start_encrypted_hmac_session</code>). Salt key: an ECC primary in the endorsement hierarchy.	Seal/unseal of rootfs, overlay, and data FDE keys, OTA re-seal, VPN PKCS#11 PIN, VPN PSK/PPK
PKCS#11 token	<code>libtpm2_pkcs11</code> salted + bound HMAC sessions. Token and key provisioning goes through the library, not <code>tpm2_ptool</code> .	Token wrapping key, key generation, signing

Verified on the device:

- `/sys/class/tpm/tpm0/null_name` exists.
- Kernel tracing shows `tpm_buf_fill_hmac_session` and `tpm_buf_check_hmac_response` on every `hwrng` TPM command.

- The kernel keeps its session open between hwrng calls. When user space sends a TPM command, the kernel flushes its session first and opens a new one on the next hwrng call.
- No HMAC or null-key errors in dmesg.

Cost: about 24 ms per 32-byte `TPM2_GetRandom` with the session, against about 9 ms without (128 commands measured). At idle rates the impact is negligible.

4. Evaluation notes and limits

- **FIPS boundary.** The kernel's session crypto (ECDH P-256, AES-128-CFB, HMAC-SHA-256) is Linux kernel crypto, outside the OpenSSL FIPS module boundary. It protects the transport only; no key material is derived from it. Key material comes from §1.1.
 - **Threat model: passive only.** The sessions defeat passive bus sniffing. Resisting an active interposer would require certifying the null primary (`null_name`) or the user-space salt key against the TPM EK certificate; this is not implemented. The kernel does refuse to use the TPM if the null primary's name changes after boot.
 - **Out of scope: the bootloader.** TF-A and U-Boot measured-boot PCR extends happen before Linux and do not use sessions.
 - **Kernel session hash.** The kernel session is fixed to SHA-256 by the kernel code. User-space sessions use a SHA-384 name-algorithm salt key.
-

Conclusion

- Key, salt, and PIN material comes from the OpenSSL FIPS CTR_DRBG. It is drawn only after the kernel CRNG has been seeded, and generation fails closed otherwise.
- The CRNG is seeded early in boot from the TPM hardware RNG (quality 1024). `rngd` and kernel timing/interrupt sources replenish it.
- Random data and secrets crossing the TPM SPI bus are encrypted on both the kernel and the user-space paths, against passive sniffing.